



CatFishing

DAM

Ainhoa Fernández Vadillo

Tutor: Carlos Rufiángel

Ainhoa Fernández Vadillo

ÍNDICES

ÍNDICES	3
ABSTRACT	6
JUSTIFICACIÓN DEL PROYECTO	7
MOTIVO	7
ANTECEDENTES	8
INTRODUCCIÓN	9
OBJETIVOS	10
DESCRIPCIÓN	18
Escalabilidad	18
Seguridad	18
Diagrama Arquitectura de la Solución	18
CU01: Carga de partida	19
CU02: Pescar	20
CU03: Dialogar con gatos	21
CU04: Comprar / Vender en el mercado	22
CU05: Comprar en la taberna	23
CU06: Interactuar con la Iglesia	25
CU07: Consultar colección	26
CU08: Gestionar Inventario	28
DISEÑOS	30
Diagrama de clases	30
Diagrama E/R	31
DIAGRAMA DE LA BASE DE DATOS	32
Diagrama de flujo de navegación	34
Interfaces	35
Main Menu	35
Configuración	35
Login y Register	36
Save Slots	37

Minijuego pesca	37
Interfaz principal con nivel, dinero, inventario manual	38
Inventario (Mochila)	38
Mercado.....	39
Taberna.....	39
Iglesia	40
Colección Pestañas Gatos y Peces	41
Mensajes UI Interactivos.....	42
Diálogos	42
TECNOLOGÍAS.....	43
METODOLOGÍA	45
Justificación y Fases:	45
Diagrama de Gantt.....	46
Comparativa de Tiempos y Desviaciones	47
Presupuesto.....	48
README y GIT	49
TRABAJOS FUTUROS	50
Optimización de la Arquitectura de Datos (Patrón Repository)	50
Inyección de Dependencias.....	50
Sistema del ciclo Día/Noche	50
Sistema de Misiones Avanzadas	51
Sostenibilidad y Reciclaje	51
Edificio "Casa de Afinidad"	51
DESPLIEGUE	52
Firebase	52
Autenticación:	52
Unity 3D.....	52
Requisitos Mínimos del Sistema	53
Manual de Instalación y Usuario.....	53
Plan de Contingencia.....	54
CONCLUSIONES	55

REFERENCIAS.....	56
------------------	----

ABSTRACT

ES

CatFishing es un videojuego 3D de simulación desarrollado como Trabajo de Fin de Grado para el ciclo de Desarrollo de Aplicaciones Multiplataforma (DAM). El juego consiste en explorar una isla, pescar diferentes tipos de peces e interactuar con diversos gatos. El objetivo principal del jugador es ir completando su colección en un álbum personalizado, mejorar la afinidad con los gatos proporcionándoles su pez favorito, y evolucionar dentro del juego mediante edificios con los que interactúa, subir de nivel y ganar dinero, nuevos ítems, buffs. A nivel técnico, el juego está conectado a una base de datos en la nube que gestiona el registro de usuarios, guarda el progreso de las partidas y carga dinámicamente los elementos del juego, ofreciendo una experiencia completa, intuitiva y en constante evolución.

EN

CatFishing is a 3D simulation game developed as Final Degree Project for the Multiplatform Application Development cycle. The game consists in exploring an island and fishing different types of fish to become friends with cats. The main objective for the player is to complete their collection of fish and improve their affinity with the cats by giving them their favourite ones, while making progress through leveling up, gaining money and trading it for more quality items. On a technical level, the game is connected to a cloud database that manages user registration, saves game progress, and loads game elements, offering a complete, intuitive, and constantly evolving experience.

JUSTIFICACIÓN DEL PROYECTO

MOTIVO

La motivación principal de este proyecto nace de mi propia experiencia como jugadora. Hay juegos en el mercado que me han inspirado mucho, como Slime Rancher por su recolección y descubrimiento de criaturas, o Stardew Valley. De este último, su mecánica de pesca transmite una tranquilidad y una sensación de farneo que me encanta.

Sin embargo, sentía que faltaba algo. Quería crear un juego en el que se evolucione constantemente, pero donde a la vez se pueda ampliar una amistad más cercana con los gatos. Buscaba generar una conexión especial cuidando de ellos, parecida a la afinidad que se crea en el juego Animal Crossing.

Por eso, me estoy esforzando mucho en el diseño y en los diálogos de cada gato y cada pez. Interactuar con los personajes es algo que me apasiona, y quiero despertar en el jugador esas ganas de ver qué personaje es el próximo que aparecerá en la isla. Ese es exactamente el sentimiento que a mí me impulsa a jugar y a engancharme a una historia.

Como todos estos juegos me gustan, quería hacer la combinación perfecta a mi gusto, porque nunca he visto en el mercado algo exactamente así.

El público al que va dirigido son jugadores que busquen un juego cozy (tranquilo), mezclado con farneo, recolección y a la vez cariño por los personajes que va descubriendo.

ANTECEDENTES

Para el desarrollo de CatFishing, se han analizado diversos títulos del género cozy y simulación (mencionados en el **Motivo**) que han servido de base para definir las mecánicas. A continuación, se presenta una comparación:

- **Stardew Valley:** Es la referencia principal para la mecánica de pesca. Aunque ofrece una experiencia de "farmeo" muy completa, la interacción con los animales es más general y no se centra específicamente en la creación de vínculos profundos mediante gustos culinarios detallados.
- **Animal Crossing:** De este juego se basa el sistema de afinidad y la importancia de los diálogos. Sin embargo, al ser un juego de gestión vecinal amplia, no profundiza en el minijuego de pesca como una mecánica de habilidad y progresión técnica tan marcada como la que se plantea.
- **Slime Rancher:** Inspira en la parte de recolección y descubrimiento de nuevas especies. Su enfoque es más de acción y exploración rápida, mientras que CatFishing busca un ritmo más pausado y centrado en la narrativa de los personajes (gatos).

Característica	Stardew Valley	Animal Crossing	Slime Rancher
Minijuego Pesca	✓	≈	✗
Sistema Afinidad	≈	✓	✗
BD Nube	✗	✗	✗
Niveles	✓	✗	✓
Cozy	✓	✓	≈

INTRODUCCIÓN

El proyecto es un juego 3D el cual consiste en pescar en una isla diversos peces, para distintos tipos de gatos. Cuando se entra al juego, el usuario tendrá un sistema de registro e inicio de sesión, ya que todo el progreso se guarda en una base de datos en la nube. A partir de ahí, el jugador podrá elegir 1 de 3 espacios de partidas guardadas, y si no hay ninguna existente, generar una nueva partida. Cada gato tiene un pez favorito, si se lo proporcionas, la afinidad con cada gato aumentará.

Se brinda al usuario un álbum con todos los peces que consigue, y los gatos con los que se ha encontrado, además de la afinidad con cada uno.

Cuanto más se pesca, más se evoluciona en el juego, se ganan más recompensas y aumenta la dificultad.

El juego cuenta con una mecánica de misiones y un mercado con moneda, donde todos los ítems y servicios se cargan desde la base de datos. En este mercado, el jugador podrá adquirir mejoras, como más espacio en el inventario, mejores cañas de pescar, cebos...

Para que la navegación sea intuitiva, se ha añadido un sistema que muestra pequeños diálogos o indicadores en pantalla (como "pulsar espacio") únicamente cuando el jugador se acerca a los objetos o gatos interactivos.

La mecánica al pescar es un minijuego dentro del propio juego, en el que hay que aguantar el cursor verde con los movimientos del pez durante un tiempo determinado.

Por último, se plantea implementar a futuro asignaturas como Sostenibilidad, para enseñar al jugador a implementar el reciclaje, craftear sus propios recursos y cuidar el medio ambiente de la isla, además de nuevas funcionalidades como misiones, interacción con más edificios...

OBJETIVOS

- **R01.** Guardar todos los gatos (tipos, afinidad), peces (rarezas), materiales, progreso del usuario.
 - **R01F01.** Base de Datos funcional en Firebase (Firestore).
 - **R01F01T01.** Hacer diagramas de clases, entidad relación y base de datos.
 - **R01F01T01P01.** Validar con tutor.
 - **R01F01T02.** Crear BD.
 - **R01F01T02P01.** Insertar datos de prueba.
 - **R01F01T03.** Relaciones, vistas, acceso a datos personalizado.
 - **R01F01T03P01.** Hacer consultas que devuelven lo buscado.
 - **R01F02.** Datos base clasificados.
 - **R01F02T01.** Insertar datos de los gatos y los peces.
 - **R01F02T01P01.** Comprobar que los datos son correctos utilizando las consultas personalizadas.
 - **R01F02T02.** Insertar datos de los ítems/servicios del mercado.
 - **R01F02T02P01.** Comprobar que los datos son correctos.
 - **R01F03.** Conexión de la BD a Unity.
 - **R01F03T01.** Registrar el proyecto Unity en Firebase.
 - **R01F03T01P01.** Verificar registro.
 - **R01F03T02.** Instalar el SDK en Unity.
 - **R01F03T02P01.** Comprobar instalación exitosa.
 - **R01F03T03.** Configuración del SDK con el archivo de Firebase.

- **R01F03T03P01.** Comprobar conexión final.
- **R01F04.** Autenticación y gestión de cuentas de usuario.
 - **R01F04T01.** Implementar interfaz gráfica de registro e inicio de sesión.
 - **R01F04T01P01.** Verificar que los datos introducidos en los campos de texto se recogen correctamente y hay manejo de errores (ej. contraseña corta).
 - **R01F04T02.** Conectar el sistema de autenticación con la BD.
 - **R01F04T02P01.** Comprobar que se crea el usuario en la BD correctamente.
 - **R01F04T03.** Vincular la partida guardada (peces, gatos, inventario) al ID del usuario activo.
 - **R01F04T03P01.** Verificar que al iniciar sesión se cargan los datos de ese jugador y no los de otro.
- **R01F05.** Pruebas automatizadas con NUnit.
 - **R01F05T01.** Crear clases Test de lo que queremos automatizar.
 - **R01F05T01P01.** Verificar que las pruebas se cumplen correctamente y devuelven lo solicitado.

- **R02.** Entorno de juego, físicas y control de personaje.
 - **R02F01.** Isla.
 - **R02F01T01.** Crear colliders de la isla.
 - **R02F01T01P01.** Verificar que los colliders bloquean correctamente.
 - **R02F01T02.** Crear iluminación de la isla.
 - **R02F01T02P01.** Comprobar que todas las luces reflejan correctamente.
 - **R02F01T03.** Generar agua.
 - **R02F01T03P01.** Comprobar que el agua está animada correctamente y las texturas cargan bien.
 - **R02F02.** Movimiento del jugador.
 - **R02F02T01.** Script de movimiento del jugador fluido.
 - **R02F02T01P01.** Comprobar que el movimiento es fluido y todas las teclas hacen lo establecido.
 - **R02F03.** Música y sonidos ambientales.
 - **R02F02T01.** Establecer música de fondo.
 - **R02F02T01P01.** Comprobar que la música se reproduce correctamente.
 - **R02F02T01.** Generar sonidos de pasos, pesca, gatos.
 - **R02F02T01P01.** Comprobar que los sonidos se reproducen cuando deben (al interactuar, caminar, pescar).

- **R03.** Interacción personalizada con cada gato.
 - **R03F01.** Spawn de los gatos en la isla.
 - **R03F01T01.** Crear Prefabs de los distintos tipos de gatos con sus características personalizadas.
 - **R03F01T01P01.** Verificar que el Prefab tiene los componentes necesarios (Collider, Animator, Script).
 - **R03F01T02.** Establecer puntos de Spawn de los gatos en la escena.
 - **R03F01T02P01.** Comprobar que los gatos aparecen en los puntos establecidos y con un % de aparición correcto.
 - **R03F02.** Diálogo de cada gato con el jugador.
 - **R03F02T01.** Implementar sistema de tocar X para interactuar
 - **R03F02T01P01.** Verificar que aparece el icono de "Interactuar" al acercarse y funciona.
 - **R03F02T02.** Crear base de datos de textos (ScriptableObject o JSON) para los diálogos.
 - **R03F02T02P01.** Comprobar que el texto mostrado corresponde al gato.
 - **R03F03.** Comercio con el gato.
 - **R03F03T01.** Entregar el pez favorito al gato.
 - **R03F03T01P01.** Verificar que se entrega bien el pez y el gato devuelve el diálogo correspondiente.
 - **R03F03T02.** Lógica del inventario y dinero.
 - **R03F03T02P01.** Verificar que al comerciar cambian los datos.

- **R03F04.** Afinidad jugador-gato.
 - **R03F04T01.** Crear "Amistad" con el Gato.
 - **R03F04T01P01.** Verificar la afinidad sube/baja.
 - **R03F04T02.** Programar reacciones del Gato.
 - **R03F04T02P01.** Comprobar que el Gato se pone feliz/triste y devuelve el diálogo correcto.
- **R04.** Evolución constante del jugador.
 - **R04F01.** Subir de nivel.
 - **R04F01T01.** Definir tabla de experiencia (XP por nivel).
 - **R04F01T01P01.** Simular ganar xp y verificar subir de nivel.
 - **R04F01T01.** Recompensas por nivel.
 - **R04F01T01P01.** Verificar que subir de nivel se ganan recompensas.
 - **R04F02.** Mercado interactivo.
 - **R04F02T01.** Establecer precios base de los ítems/servicios.
 - **R04F02T01P01.** Verificar el cálculo de venta / compra.
 - **R04F02T02.** Establecer venta.
 - **R04F02T02P01.** Comprobar que la venta es exitosa y el dinero sube.
 - **R04F02T03.** Establecer compra.
 - **R04F02T03P01.** Comprobar que el dinero e inventario se actualizan.

- **R04F03. Taberna.**
 - **R04F03T01.** Establecer precios bases de los consumibles.
 - **R04F03T01P01.** Verificar que se consumen correctamente.
 - **R04F03T02.** Establecer % de efecto en los consumibles.
 - **R04F03T02P01.** Verificar que las estadísticas se cambian correctamente durante el tiempo establecido.
- **R04F04. Iglesia.**
 - **R04F04T01.** Establecer racha de días y bufos que se consiguen.
 - **R04F04T01P01.** Comprobar que se cuentan los días y los bufos se cumplen.
- **R04F05. Colección del jugador**
 - **R04F05T01.** Crear lista de lo que descubre el jugador
 - **R04F05T01P01.** Comprobar que se va desbloqueando lo que consigue.

- **R05.** Interfaz de usuario clara e intuitiva
 - **R05F01.** Menús
 - **R05F01T01.** Crear Menú principal
 - **R05F01T01P01.** Verificar que los botones pulsados funcionan
 - **R05F01T02.** Crear Menú opciones
 - **R05F01T02P01.** Verificar que las opciones se actualizan
 - **R05F01T03.** Crear Menú taberna.
 - **R05F01T03P01.** Verificar que los estados se actualizan
 - **R05F01T04.** Crear Menú colección
 - **R05F01T04P01.** Verificar que la colección se actualiza
 - **R05F01T05.** Crear Menú mercado
 - **R05F01T05P01.** Verificar que el mercado confirma las transacciones.
 - **R05F01T06.** Crear UI interactiva.
 - **R05F01T06P01.** Verificar que los diálogos temporales e interfaz de niveles y dinero se actualizan.
 - **R05F01T07.** Diálogos
 - **R05F01T07P01.** Verificar que los diálogos se muestran e interactúan correctamente.
 - **R05F02.** Diálogos de instrucciones.
 - **R05F02T01.** Sistema de proximidad con triggers entre el player y lo que interactúa.
 - **R05F02T01P01.** Comprobar que se detecta al player dentro de los triggers.

- **R05F02T02.** Mostrar y ocultar elemento de interfaz x encima (pulsar espacio para interactuar)
 - **R05F02T02P01.** Verificar que el texto aparece y desaparece correctamente.
- **R06.** Interacción con los edificios personalizada
 - **R06F01.** Edificio mercado
 - **R06F01T01.** Crear interacción con el edificio
 - **R06F01T01P01.** Verificar que las compras/ventas se realizan correctamente.
 - **R06F02.** Edificio iglesia
 - **R06F02T01.** Crear interacción con el edificio
 - **R06F02T01P01.** Verificar que proporciona boosts y se acumula la racha.
 - **R06F03.** Edificio taberna
 - **R06F03T01.** Crear interacción con el edificio
 - **R06F03T01P01.** Verificar que proporciona boosts, gasta bien el dinero y actualiza estados del jugador.

DESCRIPCIÓN

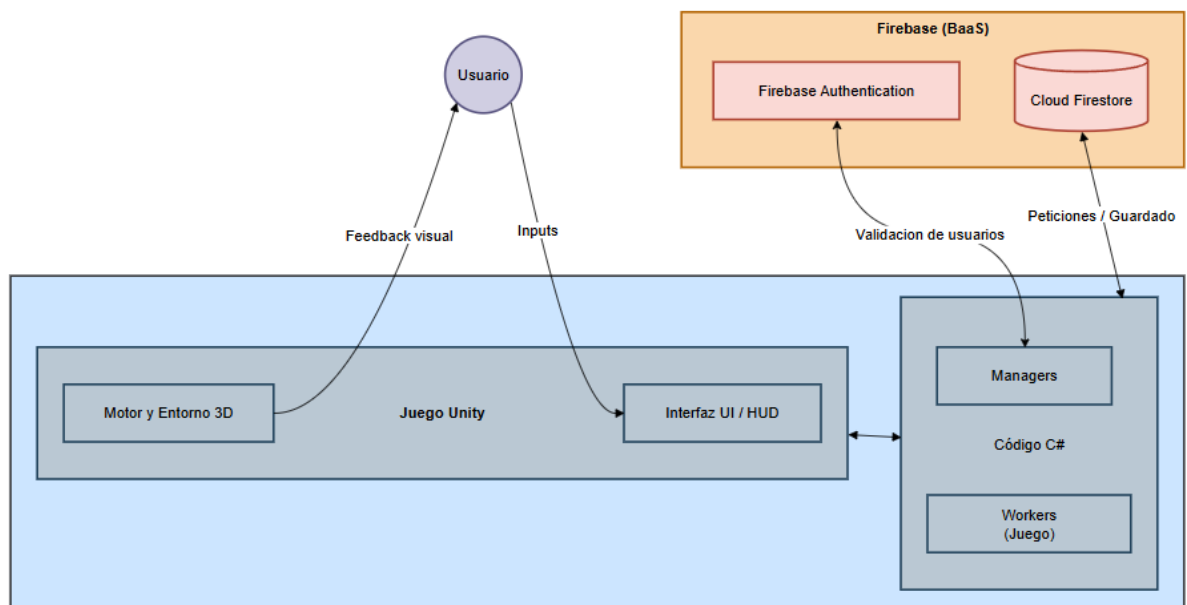
ESCALABILIDAD

Para garantizar que el juego pueda crecer en el futuro sin problemas de rendimiento, se utiliza Firebase como Backend as a Service (BaaS). Al utilizar una arquitectura NoSQL (Almacena Datos), el juego puede crecer añadiendo cientos de peces o gatos nuevos en la base de datos sin necesidad de recompilar el código de Unity.

SEGURIDAD

El acceso a la base de datos está protegido mediante Firebase Authentication, validando los inputs del usuario (contraseñas seguras, formatos de email válidos mediante Regex) desde el propio cliente antes de procesar las peticiones, lo que evita inyecciones de datos malignos en la BD.

DIAGRAMA ARQUITECTURA DE LA SOLUCIÓN



CU01: CARGA DE PARTIDA

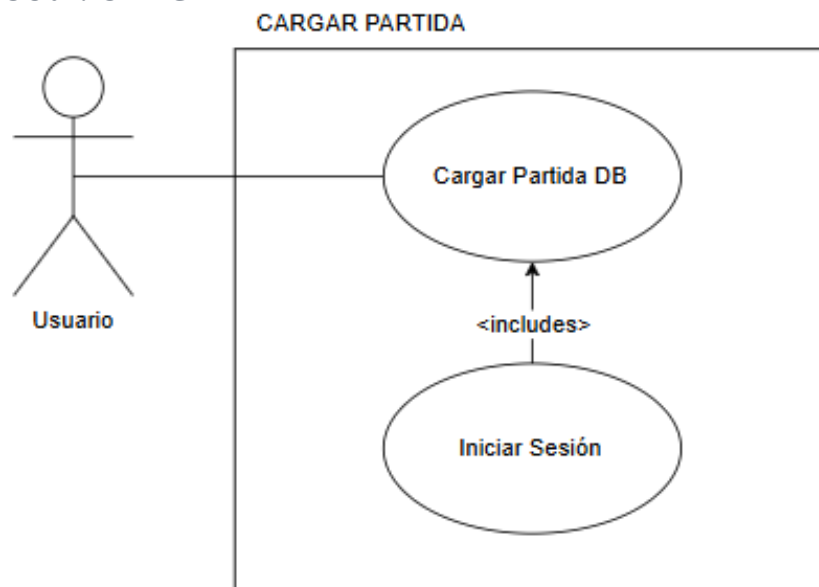


Ilustración 1: CU01 Carga de partida.

DESCRIPCIÓN 1. El jugador introduce user y contraseña en la pantalla de inicio. 2. El sistema valida sus datos en la bd. 3. Se descargan los datos del slot de guardado de la partida. 4. El jugador entra en su partida.	
PRECONDICIONES Usuario registrado y conexión a Internet.	POSTCONDICIONES Sesión activa del usuario.
DATOS ENTRADA Email bd. Contraseña bd.	DATOS SALIDA Menú Login/Registro marca inicio correcto. Carga de los datos del slot de partida.
BD users users/save_slots	CLASES AuthManager.cs MenuManager.cs
INTERFACES: UI Login/Registro menú principal.	

Tabla 1: CU01 Carga de partida.

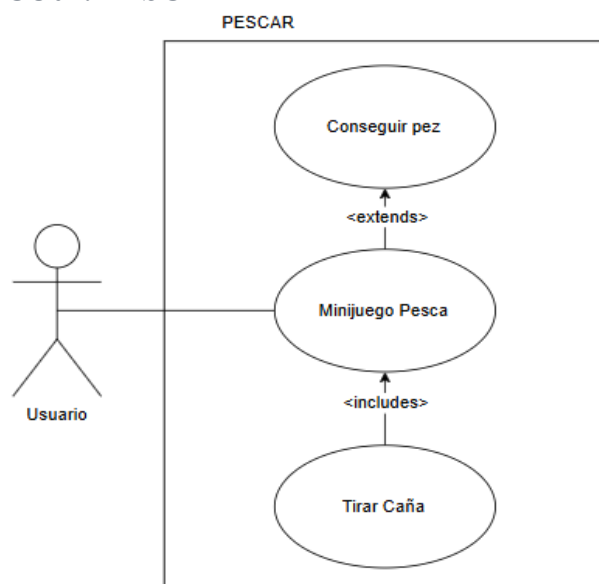
CU02: PESCAR

Ilustración 2: CU02 Pescar.

DESCRIPCIÓN 1. El jugador presiona espacio en la zona de pesca. 2. Entre un tiempo aleatorio, aparece la ui del minijuego. 3. El jugador mantiene la barra verde con el pez. 4. Gana/Pierde. Si gana, se calcula la rareza, se genera el pez y se guarda en Firebase.	
PRECONDICIONES Jugador en trigger zona de pesca.	POSTCONDICIONES Guardar pez en el inventario y colección.
DATOS ENTRADA Inputs click del jugador en el minijuego.	DATOS SALIDA Si gana, recoge con el pez. Si pierde, pez escapado.
BD fish users/save_slots	CLASES PescaController.cs ZonaPesca.cs FishManager.cs
INTERFACES UI Minijuego pez, panel de captura.	

Tabla 2: CU02 Pescar.

CU03: DIALOGAR CON GATOS

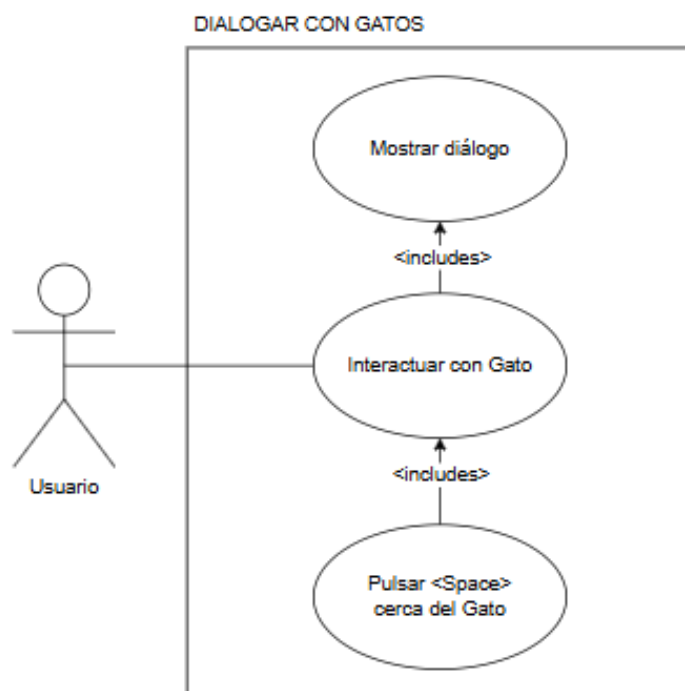


Ilustración 3: CU03 Dialogar con gatos.

DESCRIPCIÓN 1. El jugador entra en el trigger del gato y le da al espacio. 2. El sistema identifica x ID al gato y se cargan sus datos. 3. Se muestran los paneles de diálogo y se interactúa hasta que termine la conversación.	
PRECONDICIONES Jugador en trigger del gato.	POSTCONDICIONES Cargar el diálogo de la BD.
DATOS ENTRADA Input interacción en trigger del jugador.	DATOS SALIDA Panel UI diálogos cargados de BD.
BD cats users/save_slots	CLASES GatoNPC.cs CatBehaviour.cs UIManager.cs
INTERFACES: UI diálogos.	

Tabla 3: CU03 Dialogar con gatos.

CU04: Comprar / Vender en el mercado

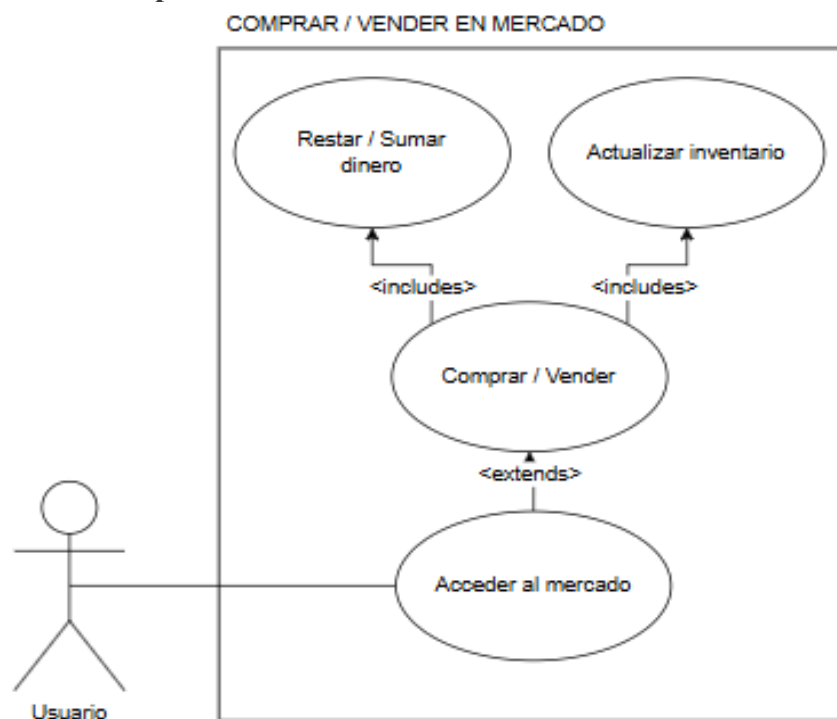


Ilustración 4: CU04 Comprar / Vender en el mercado.

DESCRIPCIÓN 1. El jugador interactúa con el edificio del mercado. 2. La interfaz muestra 2 pestañas: 1 de los ítems a comprar y 1 del inventario. 3. El jugador elige si compra o vende. 4. Se confirma la transacción y actualizan campos inventario y dinero.	
PRECONDICIONES Jugador en trigger de Mercado. Comprar: Cantidad de dinero superior al precio. Vender: Tener ítem que vender inventario.	POSTCONDICIONES Actualizar campos al comprar / vender restando / sumando el precio base y actualizando el inventario.
DATOS ENTRADA Clicks jugador, comprar/vender/seleccionar ítems, confirmar.	DATOS SALIDA Ítems aparecen/desaparecen en los menús, dinero se actualiza, sonidos compra / venta.

BD fish / items users/save_slots	CLASES BuildingInteraction.cs MarketManager.cs MarketSlot.cs InventorySystem.cs
INTERFACES: UI Mercado, UI inventario, HUD Dinero, UI Manager (alertas y tooltips).	

Tabla 4: CU05 Gestionar Misiones.

CU05: COMPRAR EN LA TABERNA

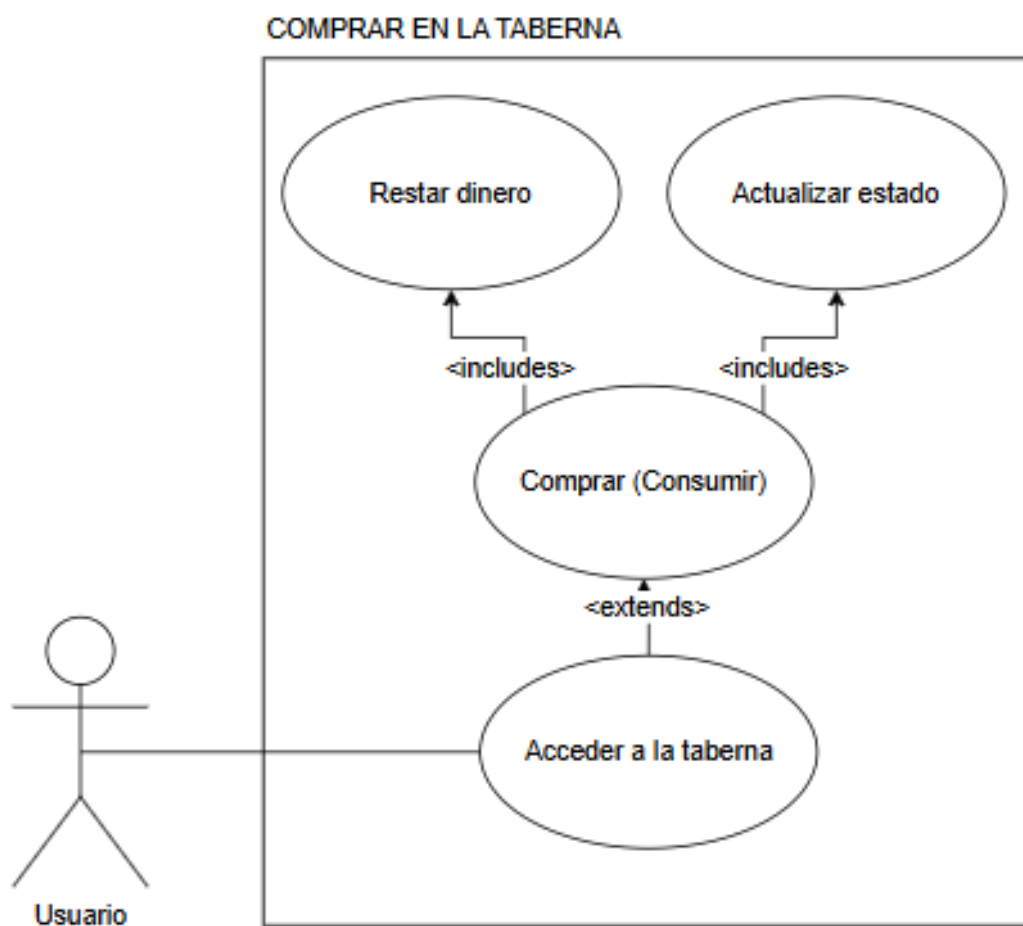


Ilustración 5: CU05 Comprar en la taberna.

DESCRIPCIÓN 1. El jugador interactúa con el edificio de la taberna. 2. La UI de la Taberna y se muestran los servicios cargados de la BD. 3. El jugador selecciona un consumible y confirma la compra. 4. Se resta el dinero y se aplica el multiplicador o bufo al GameManager.	
PRECONDICIONES Jugador en trigger de Taberna. Comprar: Cantidad de dinero superior al precio	POSTCONDICIONES Actualizar campos al comprar bajando el dinero y aplicando efectos.
DATOS ENTRADA Input del teclado para interactuar. Clicks en la UI para consumir.	DATOS SALIDA Se actualiza dinero, sonido compra, actualiza estado.
BD services users/save_slots	CLASES TavernManager.cs TavernSlot.cs ServiceData.cs GameManager.cs
INTERFACES: UI Taberna, UI Manager (alertas y tooltips), HUD Dinero.	

Tabla 5: CU05 Comprar en la taberna.

CU06: INTERACTUAR CON LA IGLESIA

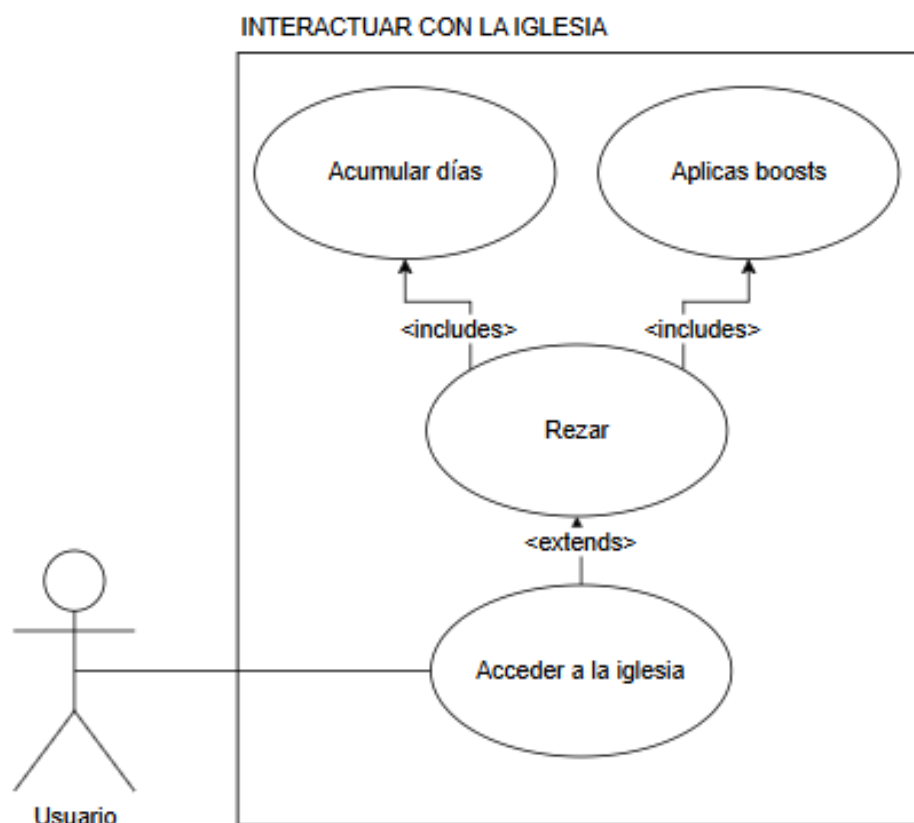


Ilustración 6: Interactuar con la Iglesia.

DESCRIPCIÓN

1. El jugador interactúa con el edificio de la iglesia.
2. La interfaz compara la última fecha del sv con la última fecha rezada.
3. El jugador pulsa el botón para rezar.
4. Se actualiza la racha en la BD.

PRECONDICIONES

Jugador en trigger de la Iglesia.
No haber rezado ese día.

POSTCONDICIONES

Actualizar campos al comprar bajando el dinero y aplicando efectos.

DATOS ENTRADA

Input del teclado para interactuar.
Boton de rezar.

DATOS SALIDA

Se actualiza el último día rezado, los bufos y +1 en la racha de días.

BD services users/save_slots	CLASES ChurchManager.cs ChurchSlot.cs GameManager.cs
INTERFACES: UI Taberna, UI Manager (alertas y tooltips), HUD Dinero.	

Tabla 6: CU06 Interactuar con la Iglesia.

CU07: CONSULTAR COLECCIÓN

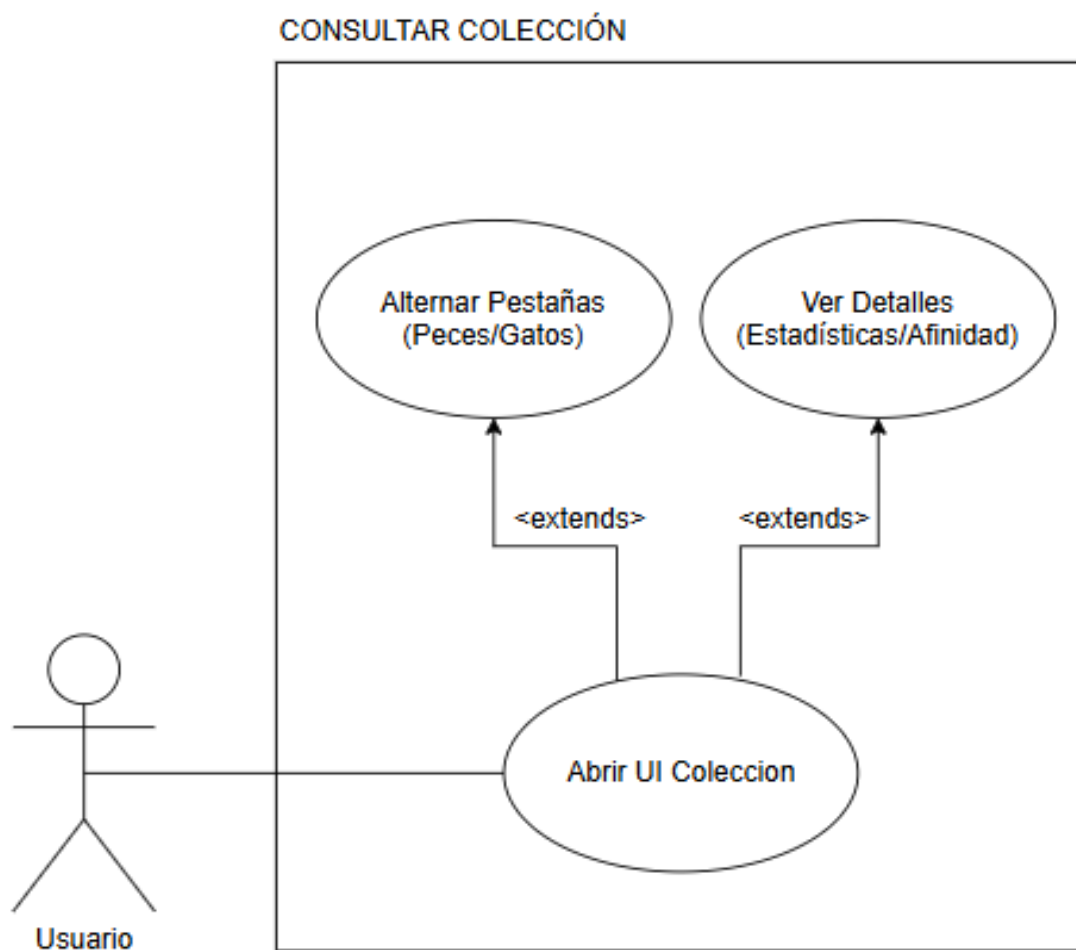


Ilustración 7: Consultar Colección

DESCRIPCIÓN 1. El jugador abre la colección (C). 2. El sistema carga el progreso del jugador en el catálogo. 3. El jugador alterna entre la pestaña de gatos y la de peces. 4. Se pueden observar los detalles de los elementos desbloqueados.	
PRECONDICIONES -	POSTCONDICIONES Actualizar campos al comprar bajando el dinero y aplicando efectos.
DATOS ENTRADA Input (C) para abrir el menú. Clicks en la UI.	DATOS SALIDA Elementos que ha desbloqueado el jugador, cargados desde la BD y sus detalles.
BD cats fish users/save_slots	CLASES CollectionManager.cs CollectionSlot.cs FishManager.cs CatData.cs ItemData.cs
INTERFACES: UI Colección, Previews de los peces y gatos.	

Tabla 7: CU07 Consultar Colección

CU08: GESTIONAR INVENTARIO

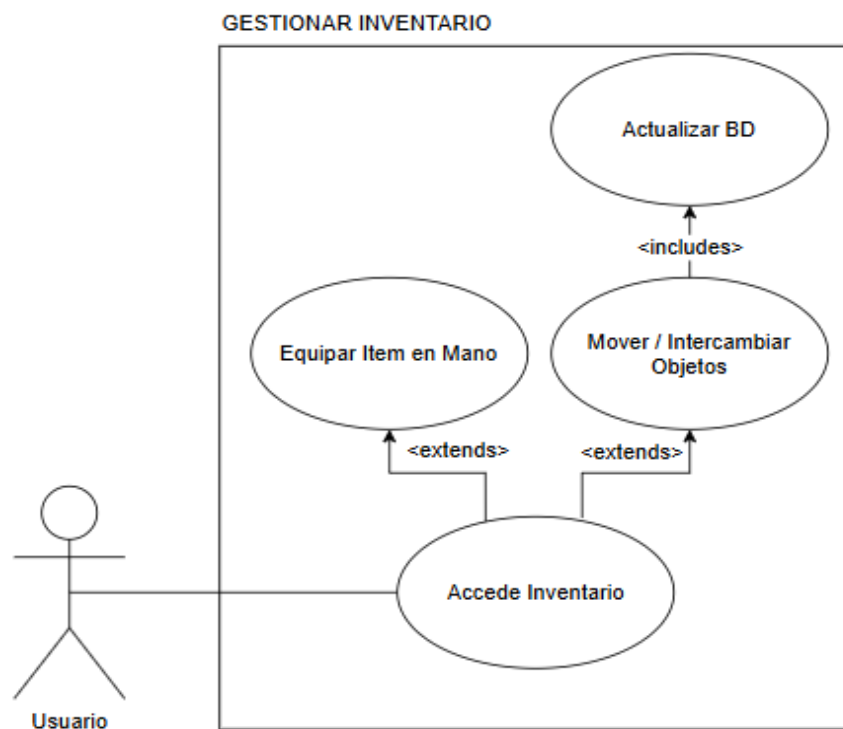


Ilustración 8: Gestionar Inventario.

DESCRIPCIÓN

1. El jugador pulsa abre la mochila completa pulsando (Tab) o (I), o usa las teclas 1-5 para la barra rápida.
2. Al hacer clic en una casilla y luego otra, los ítems se cambian de lugar.
3. Al equipar un ítem, el prefab aparece en la mano del jugador.
4. Las posiciones del inventario se guardan en la BD.

PRECONDICIONES

El inventario se ha cargado correctamente desde la BD.

POSTCONDICIONES

Actualizar campos al comprar bajando el dinero y aplicando efectos.

DATOS ENTRADA

Input (Tab) (I) para abrir la mochila.
Clicks para seleccionar o mover ítems.

DATOS SALIDA

Elementos que ha desbloqueado el jugador, cargados desde la BD, y sus detalles.

BD users/save_slots	CLASES InventorySystem.cs InventorySlot.cs
INTERFACES: UI Barra Inventario, UI Mochila.	

Tabla 8: CU08 Gestionar Inventario.

DISEÑOS

DIAGRAMA DE CLASES

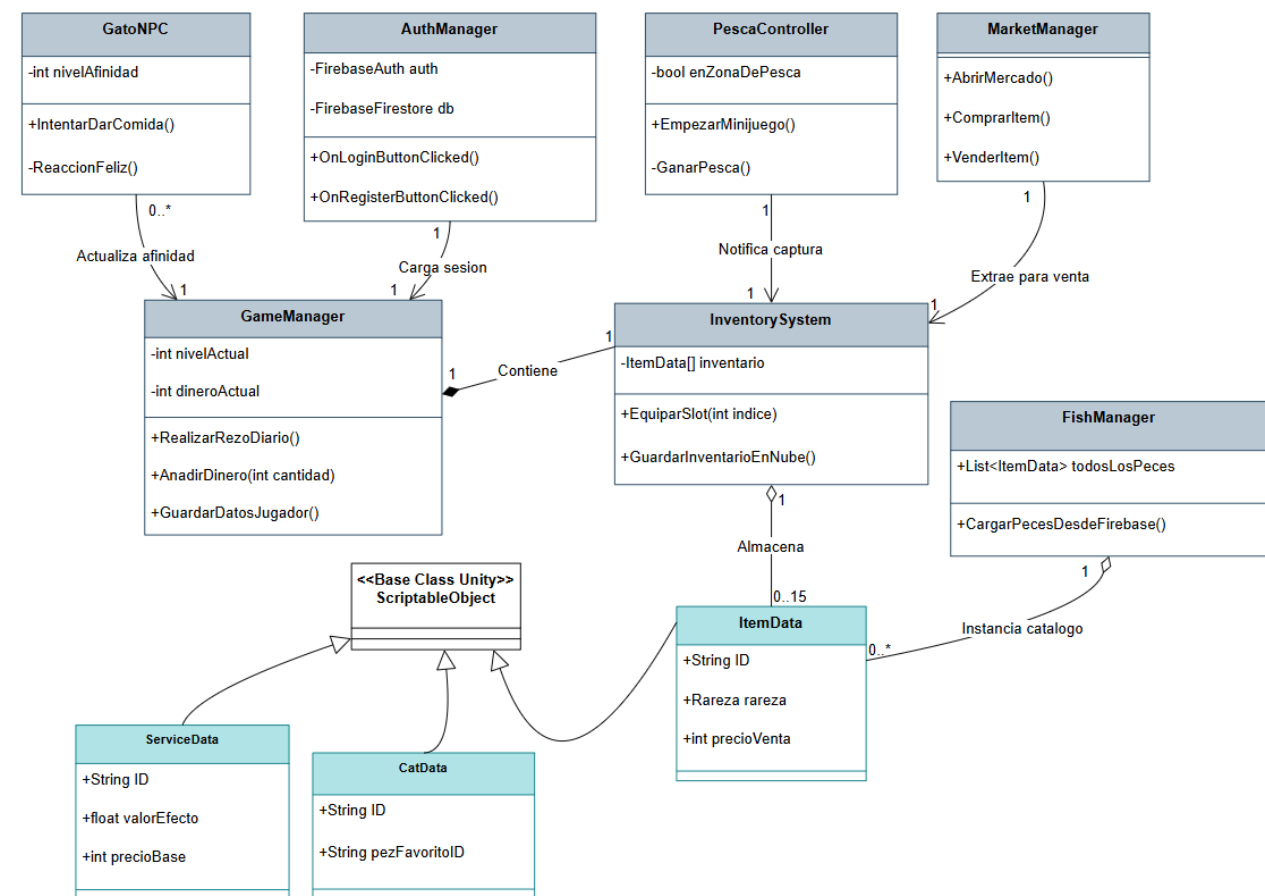


DIAGRAMA E/R

Aunque se utilice una Base de Datos No-SQL, las relaciones entre las tablas son mediante campos comunes, Foreign Keys manualmente hechas, por ejemplo, un id de Fish se encuentra en Cat como id_pez_favorito.

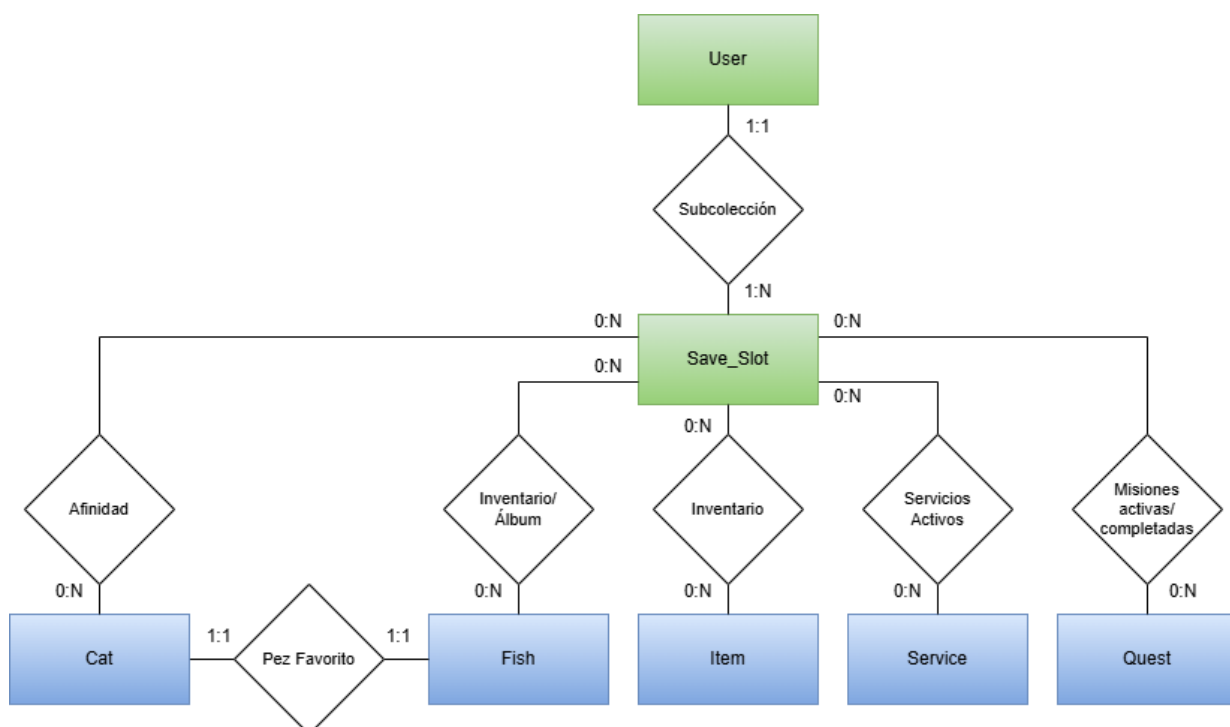
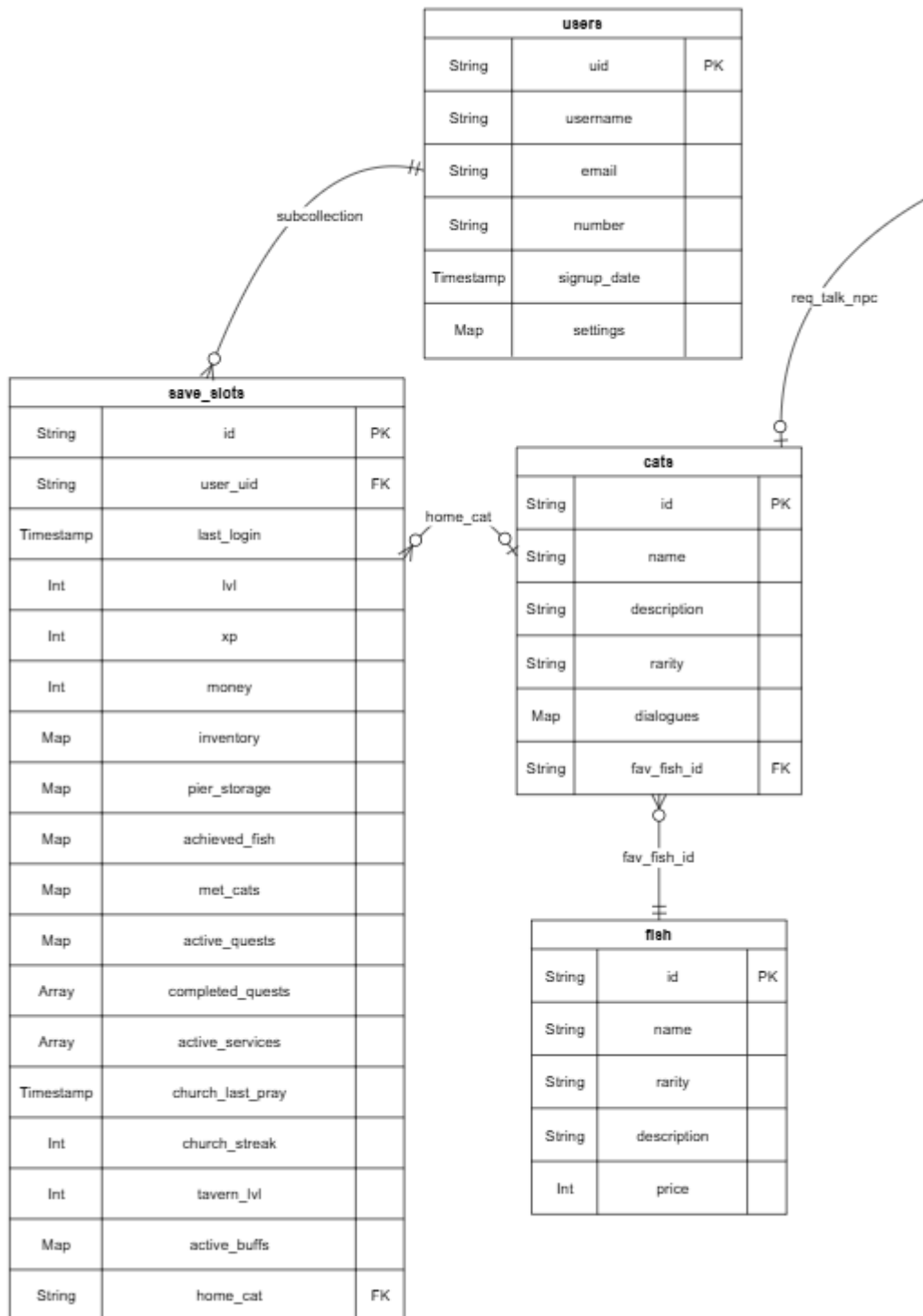


DIAGRAMA DE LA BASE DE DATOS



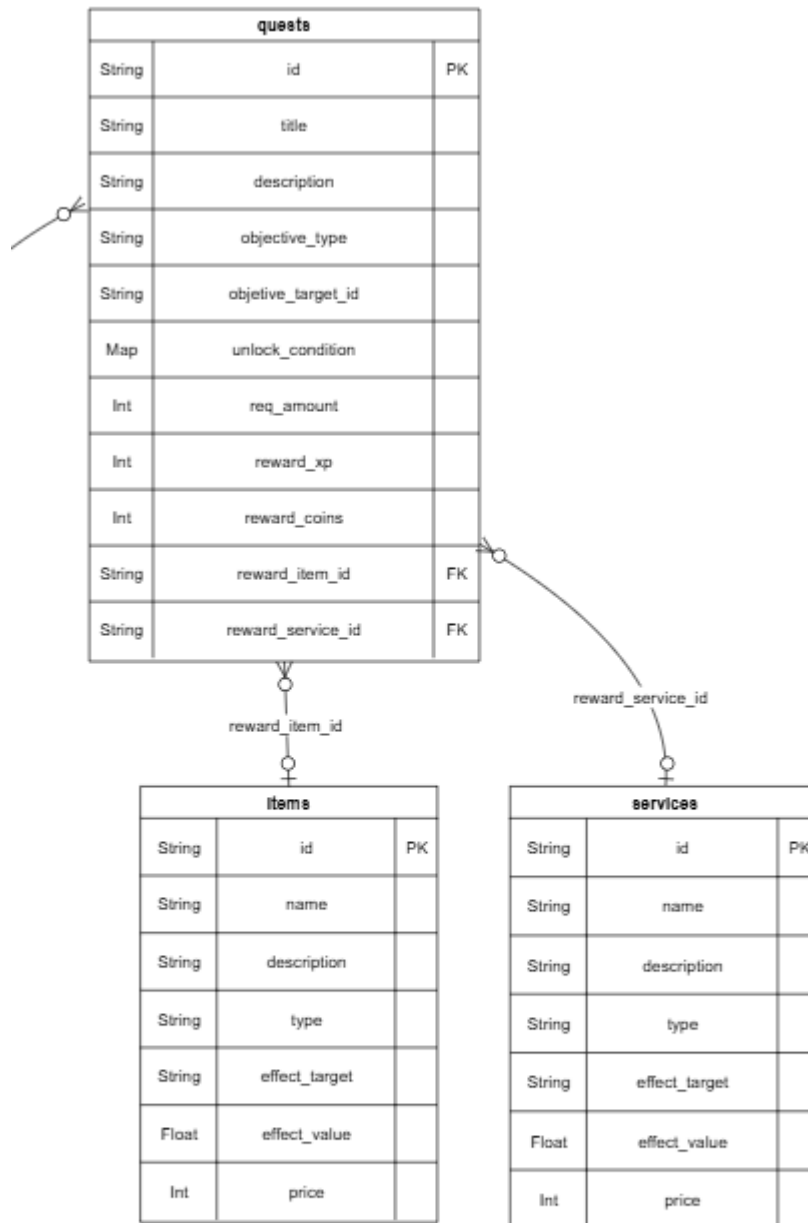
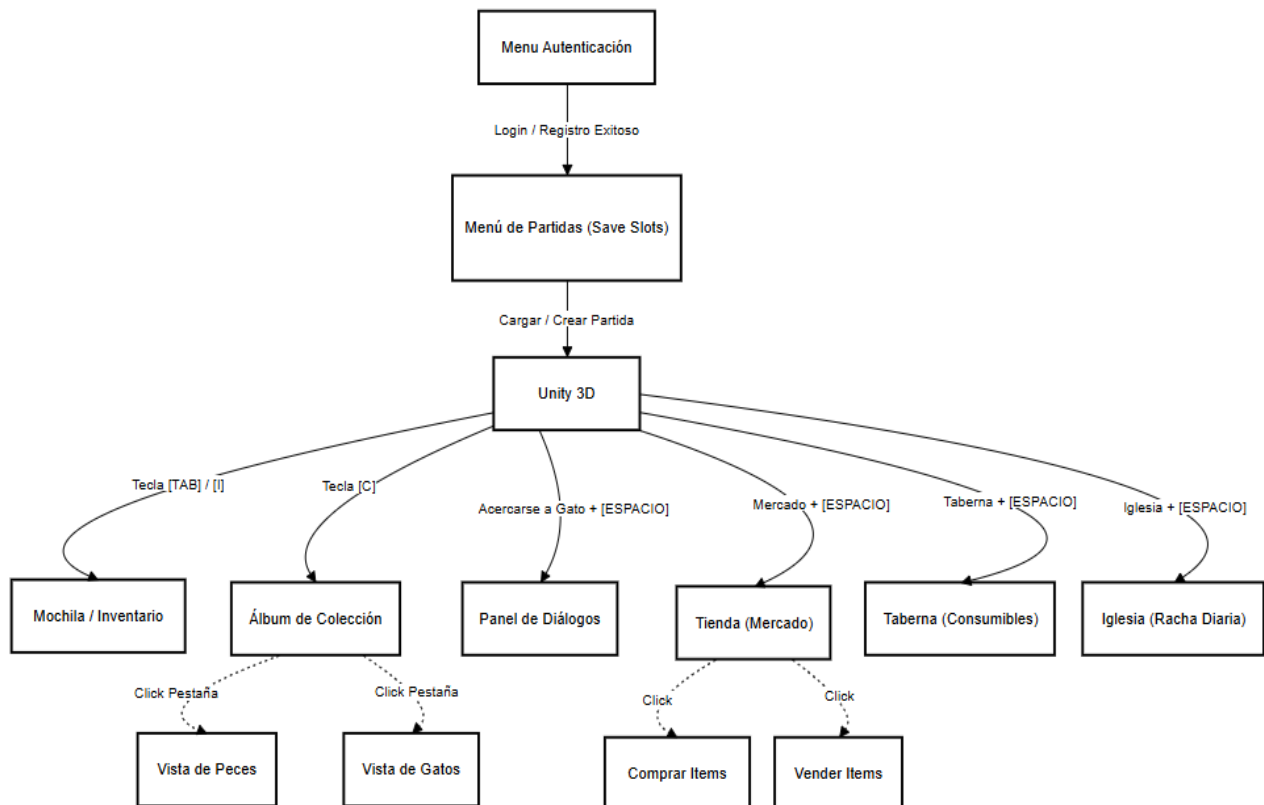
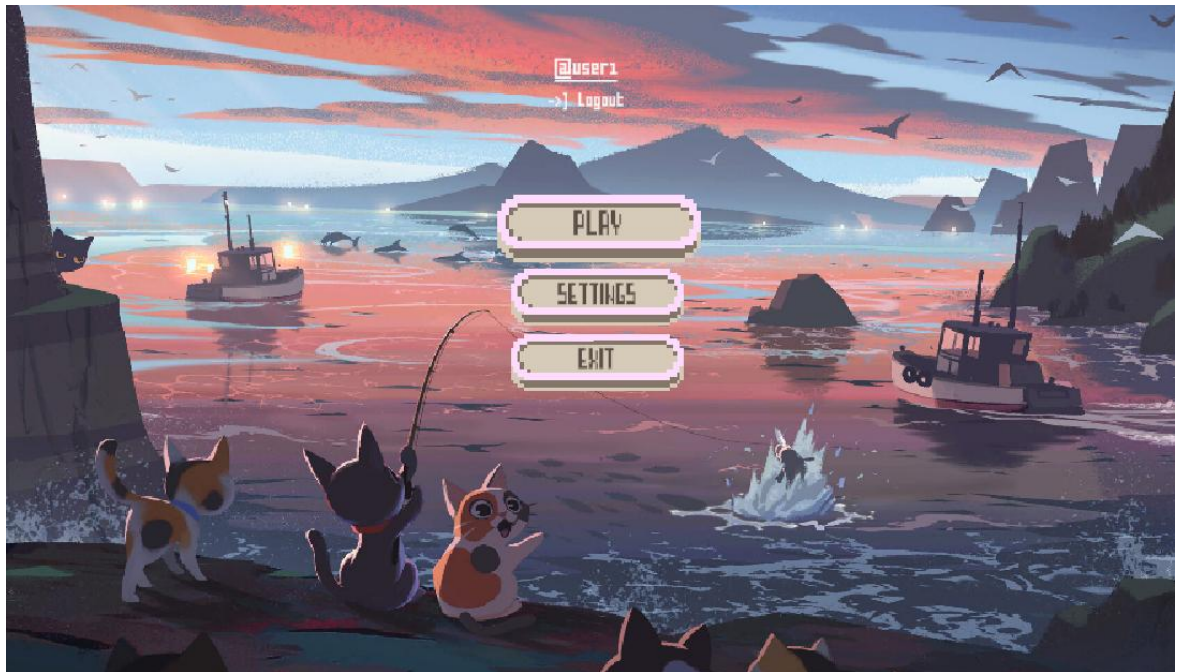


DIAGRAMA DE FLUJO DE NAVEGACIÓN



INTERFACES

Main Menu



Configuración



Login y Register

The image displays two screenshots of a web application's login and register forms, set against a pixel-art background of a boat on water at sunset with a cat on a cliff.

Top Screenshot: LOGIN

Header: **LOGIN**

Text: Por favor, rellena todos los campos.

Form fields:

-
-
- ☐ Remember me

Buttons:

- LOGIN**
- [No account? REGISTER](#)

Bottom Screenshot: REGISTER

Header: **REGISTER**

Text: El nombre de usuario y contraseña sólo pueden contener letras y números.

Form fields:

-
-
-
-

Buttons:

- REGISTER**
- [LOGIN](#)

Save Slots



Minijuego pesca



Interfaz principal con nivel, dinero, inventario manual



Inventario (Mochila)



Mercado



Taberna



Iglesia



Colección Pestañas Gatos y Peces



Mensajes UI Interactivos



Diálogos



TECNOLOGÍAS

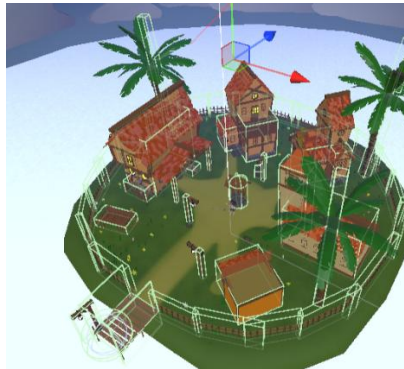
Las tecnologías y herramientas utilizadas para este proyecto.



Unity.

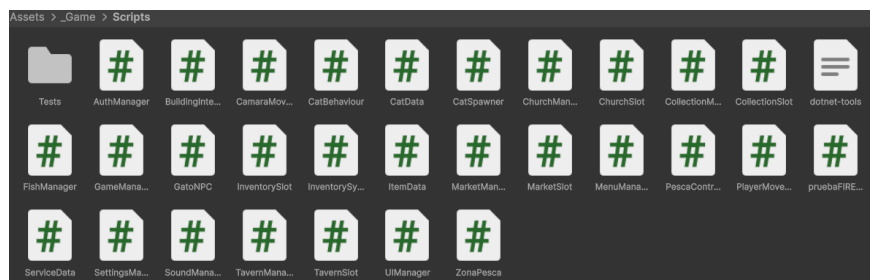
Desarrollo de videojuegos en 3D.

La estructura de la isla, personajes, jugabilidad se ha hecho con este programa.



C#.

Todo el código de este programa se ha desarrollado en C#.



Firestore - Firebase.

En este programa, todo se almacena en una base de datos Firestore en Firebase.

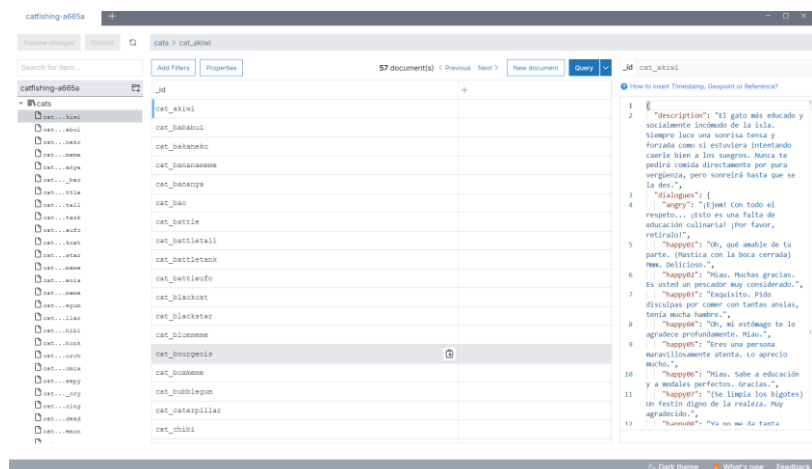
Los usuarios acceden mediante mail y contraseña validados.



Refi App - Firebase.

Programa GUI de Firebase.

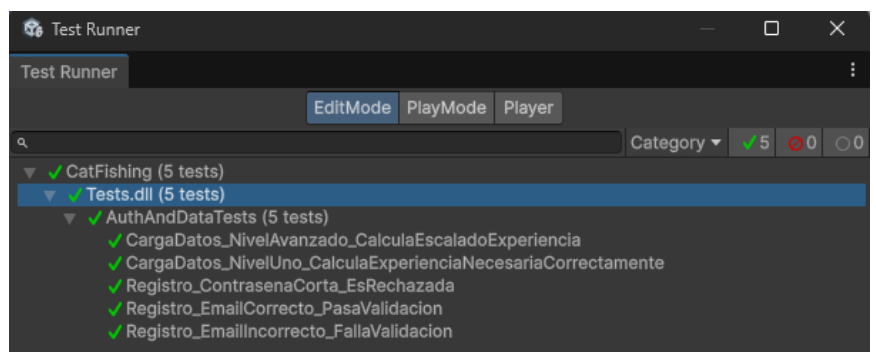
Utilizado para insertar los JSON de los personajes, ítems, servicios del juego.



NUnit.

Se ha utilizado el framework NUnit para una prueba automatizada de la validación de usuarios.

Se ejecuta desde el menú Test Runner de Unity, en un código de Test programado en C#.



METODOLOGÍA

En este proyecto se ha utilizado **metodología ágil** basada en un modelo **RFTP**, calculando las horas dedicadas a cada tarea y siguiendo una planificación estructurada.

JUSTIFICACIÓN Y FASES:

1. **Diseño y Planificación:** Se definieron los requisitos del sistema y se diseñó la arquitectura de la base de datos en Firestore.
2. **Desarrollo del Backend:** Configuración de Firebase Authentication para la seguridad y creación de las colecciones de datos (peces, gatos, usuarios).
3. **Testing:** Uso del framework **NUnit** para automatizar la validación de registros e inicios de sesión.
4. **Implementación en Unity:** Creación de las mecánicas principales (movimiento, pesca, inventario) y su conexión con Firestore mediante C#.

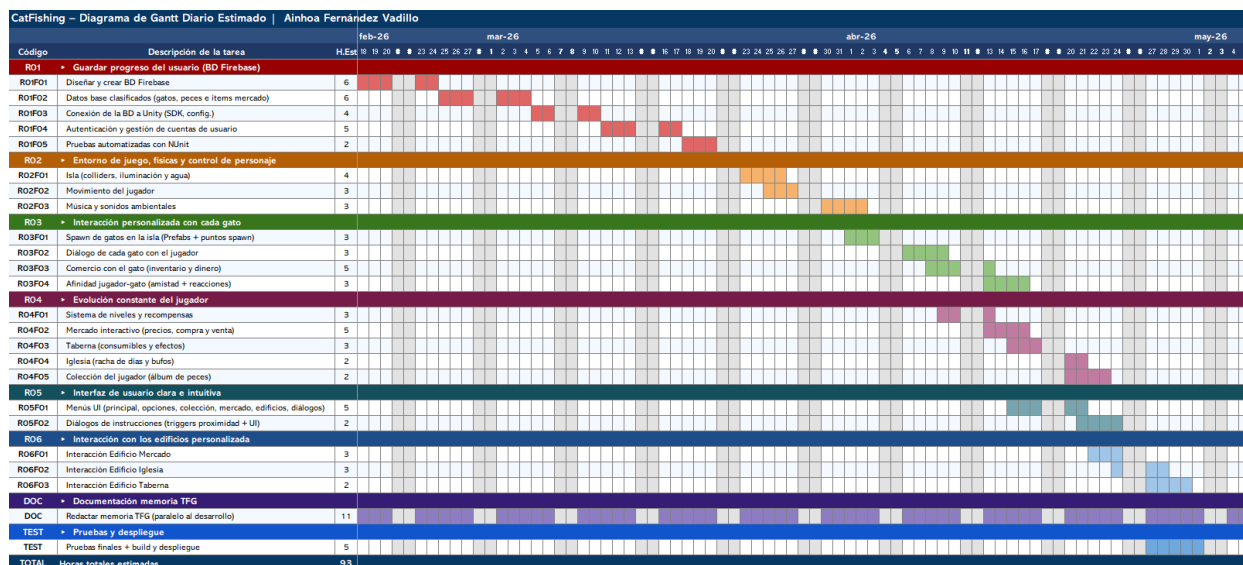
El seguimiento del tiempo se ha realizado comparando una **planificación estimada** frente al **tiempo real** dedicado:

Ainhoa Fernández Vadillo

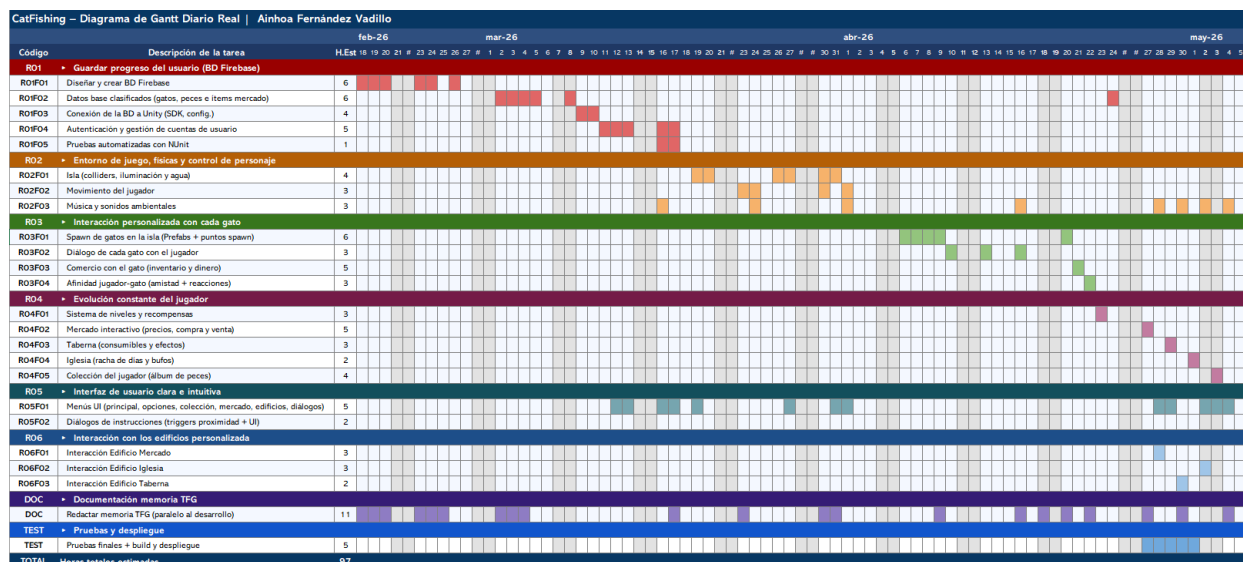
DIAGRAMA DE GANTT

Excel completo en el Repositorio de GitHub.

Previsión:



Real:



Comparativa de Tiempos y Desviaciones

El proyecto ha experimentado una desviación total de +4 horas respecto a la previsión inicial. Se puede observar que al final del tiempo establecido se ha trabajado más, puesto que en el Requisito 2 se cometió un error que supuso horas de pulimiento, y en el 4 se tardó más de lo esperado en reflejar los datos en la interfaz.

Código	Descripción de la Tarea	Horas Est.	Horas Reales	Desviación
R01F05	Pruebas automatizadas con NUnit	2	1	-1h
R03F01	Spawn de gatos (Prefabs + puntos)	3	6	+3h
R04F05	Colección del jugador (Álbum)	2	4	+2h
TOTAL	Cómputo global del proyecto	93	97	+4h

PRESUPUESTO

Requisito Principal	Horas Reales	Presupuesto (Real)
R01: Base de Datos y Seguridad	22h	550 €
R02: Entorno, Físicas y Control	10h	250 €
R03: Interacción con Gatos	17h	425 €
R04: Evolución y Sistemas de Juego	17h	425 €
R05: Interfaz de Usuario (UI/UX)	7h	175 €
R06: Lógica de Edificios	8h	200 €
Documentación y Memoria	11h	275 €
Pruebas Finales y Despliegue	5h	125 €
COSTO TOTAL DEL PROYECTO	97h	2.425 €

Ainhoa Fernández Vadillo

README Y GIT

https://github.com/inoixh/TFG_2DAM

TRABAJO FUTURO

El juego ha sido diseñado con una arquitectura modular que permite implementar nuevas mecánicas con facilidad. De cara a futuras versiones, se planean las siguientes ampliaciones:

Optimización de la Arquitectura de Datos (Patrón Repository)

Actualmente, las llamadas a la API de Firebase están integradas directamente en los controladores (AuthManager, MarketManager). Como mejora de escalabilidad, se propone implementar el **Patrón Repository**. Crear una capa intermedia (interfaz) que separe la lógica del juego de la tecnología de persistencia. De este modo, el código no interactuaría directamente con la base de datos, permitiendo migrar de Firebase a otras soluciones como SQL o MongoDB en el futuro sin necesidad de modificar la lógica de negocio del videojuego.

Inyección de Dependencias

Actualmente, el proyecto utiliza el patrón **Singleton** para acceder fácilmente a sistemas como el GameManager o el InventorySystem. Es una solución práctica y común en Unity, pero hace que las clases estén muy "pegadas" entre sí. Una mejora clave sería cambiar a la **Inyección de Dependencias (DI)**. Con esto, el código sería más independiente y profesional, lo que facilitaría hacer pruebas automáticas y ampliar el juego con nuevas funciones sin que el sistema se vuelva inestable.

Sistema del ciclo Día/Noche

Implementar un reloj interno que modifique la iluminación global de la isla. Esto hará que ciertos peces o gatos legendarios tengan una probabilidad de aparición vinculada a un horario específico.

Sistema de Misiones Avanzadas

Ampliar la UI para incluir misiones dinámicas diarias o especiales con los gatos, fomentando que el jugador inicie sesión recurrentemente para obtener recompensas únicas, además de aumentar afinidad por favores a los gatos.

Sostenibilidad y Reciclaje

Añadir una mecánica en la que, al pescar, el jugador pueda extraer "basura" (plásticos, latas) del agua. Esta basura se podrá reciclar en una máquina de la isla para obtener materiales de "crafteo", concienciando al jugador sobre el cuidado del medio ambiente marino.

Edificio "Casa de Afinidad"

Desarrollar el interior de la casa del jugador para que los gatos con los que se haya alcanzado el nivel máximo de amistad (Nivel 10) puedan visitarla o instalarse en ella de forma permanente.

DESPLIEGUE

FIREBASE

No se requiere el despliegue tradicional de un servidor. La configuración se realiza a través de la consola de Firebase:

Autenticación: El inicio de sesión es a través de correo electrónico y contraseña.

Base de Datos Firestore: Se establecen reglas de seguridad para garantizar que únicamente los usuarios autenticados puedan leer y escribir en su documento correspondiente a su id, protegiendo la integridad de las partidas guardadas de otros jugadores (Reglas de Firebase).

Datos: Se inyectan los json iniciales (peces, gatos, consumibles e ítems del mercado) en sus respectivas colecciones en la BD a través de Refi App.

UNITY 3D

La compilación del cliente se realiza desde el motor Unity hacia la plataforma Windows.

Se ajustan las opciones del Player para incluir el empaquetado de los assets (modelos, texturas y audio) de forma optimizada.

El resultado es un directorio que contiene el archivo ejecutable (CatFishing.exe), las carpetas de datos nativas de Unity y las librerías del SDK de Firebase. Este directorio se comprime en formato .zip para su distribución.

REQUISITOS MÍNIMOS DEL SISTEMA

Para garantizar un rendimiento fluido y la correcta comunicación con la base de datos, el equipo del usuario final debe cumplir con las siguientes especificaciones:

- **Sistema Operativo:** Windows 10 / Windows 11 (64 bits).
- **Procesador:** Intel Core i3-8100 / AMD Ryzen 3 1200 o superior.
- **Memoria RAM:** 4 GB (6-8 GB recomendado).
- **Tarjeta Gráfica:** Compatible con DirectX 11, 2 GB VRAM mínimo.
- **Almacenamiento:** 500 MB de espacio libre.
- **Conexión:** Conexión a Internet permanente (requerida para guardado en la nube y autenticación).

MANUAL DE INSTALACIÓN Y USUARIO

El proceso de instalación para el usuario final está diseñado para ser inmediato (Plug & Play), sin requerir instaladores de terceros:

Descarga: Obtener el archivo CatFishing.zip.

Extracción: Descomprimir el archivo en el directorio local deseado (se requiere permisos de escritura en la carpeta para los archivos temporales de Unity).

Ejecución: Hacer doble clic sobre el archivo CatFishing.exe.

Uso Inicial: Al iniciarse la aplicación, el jugador deberá crear una cuenta nueva en el panel "Register" utilizando un formato de correo válido y una contraseña de al menos 6 caracteres. Posteriormente, podrá acceder al juego desde el panel "Login".

PLAN DE CONTINGENCIA

Para mitigar posibles incidencias en el entorno de producción, se han contemplado las siguientes medidas:

Caída temporal de conexión: El código cliente gestiona las tareas asíncronas de Firebase. Si falla una petición de compra o de pesca por pérdida de red, la transacción no se consolida, evitando desincronizaciones entre la UI y la base de datos de Firestore.

Corrupción de credenciales locales: Si los datos de sesión guardado caducan o se corrompen, el sistema detecta la anomalía en el inicio, cierra la sesión automáticamente y redirige al usuario a la pantalla de validación manual.

CONCLUSIONES

La realización de **CatFishing** me ha permitido aplicar los conocimientos adquiridos en el ciclo de DAM para crear una solución multiplataforma real y escalable.

A nivel técnico, el mayor logro ha sido la integración de un motor de juegos 3D como **Unity** con una arquitectura de datos **NoSQL (Firestore)**. Esto ayuda a que el juego sea dinámico: podemos añadir contenido nuevo (más gatos o peces) directamente desde la base de datos sin necesidad de actualizar el código de la aplicación, o generar Item Data manualmente.

Profesionalmente, el proyecto refleja que es posible combinar una experiencia de usuario tranquila y atractiva con un Backend sólido. **CatFishing** no es solo un videojuego, sino una muestra de cómo la programación orientada a objetos y las buenas prácticas de arquitectura permiten crear productos escalables y listos para evolucionar con nuevas funciones, como el sistema de sostenibilidad o misiones avanzadas.

REFERENCIAS

Según las normas APA. Cada referencia se acompañará de un texto descriptivo con el apartado del proyecto asociado.

- *Aplicado como asistente técnico para la resolución de errores de dependencias de SDK, diseño de expresiones regulares (regex) para validación de datos y funciones matemáticas para % probabilidad, cálculo movimiento cámara.*

Google. (2026). Gemini [Modelo de lenguaje grande]. Recuperado de <https://gemini.google.com/>

- *Aplicado para consultar la estructura de diccionarios NoSQL y la documentación de las peticiones asíncronas de lectura y escritura.*

Google. (2024). Documentación de Firebase para Unity. Recuperado de <https://firebase.google.com/docs/unity/setup>

- *Referencia principal para la integración de la API del motor gráfico, implementación de físicas y cálculos (mathf) en el minijuego de pesca.*

Unity Technologies. (2026). Unity Scripting API [Documentación de software]. Recuperado de <https://docs.unity3d.com/ScriptReference/>

- *Utilizado como framework para los scripts de Test automatizados.*

NUnit Project. (2026). NUnit Documentation [Framework de pruebas]. Recuperado de <https://docs.nunit.org/>

- *Plataforma utilizada para el control de versiones mediante git y la protección del código ante pérdidas o roturas estructurales durante el ciclo de vida del desarrollo.*

GitHub. (2026). GitHub [Plataforma de desarrollo]. Recuperado de <https://github.com/>

- *Repositorio de recursos utilizado para la obtención de los modelos 3d (prefabs) del entorno.*

- Unity Technologies. (2026). Unity Asset Store [Plataforma de recursos digitales]. Recuperado de <https://assetstore.unity.com/>

- Sketchfab. (2026). Sketchfab [Plataforma de recursos digitales]. Recuperado de <https://sketchfab.com/>

- *Herramienta de software libre utilizada para el diseño estructurado y exportación gráfica de la arquitectura del sistema, incluyendo el diagrama de flujo de navegación y el diagrama de clases uml.*

JGraph. (2026). draw.io [Aplicación web de diagramado]. Recuperado de <https://app.diagrams.net/>